Requirement Document

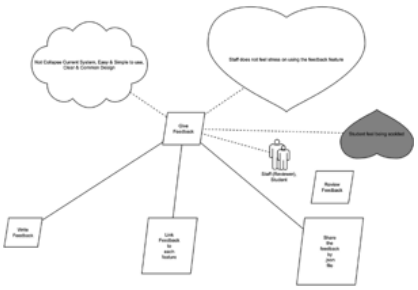
> Updated 25 Oct. All features were demonstrated to and approved by the client on 15 Oct (minutes). User stories, non-functional requirements and architecture for the guided tour and version history are on Requirements and Architecture: Guided Tour and Version History.

**1. Purpose and scope**

AMMBER is a browser-based motivational modelling tool. Models are authored client-side and exchanged as .json files; the application holds no server-side state and makes no network requests.

This document records the requirements for Weeks 1–12, distinguishing those originating with the client from those proposed by the team. Every story below matches behaviour that exists in the codebase: nothing speculative. Capabilities the team built and then deliberately withdrew are recorded in section 8 so they are not re-proposed by accident.

**2. Motivational model**



The model was produced by the team using AMMBER itself, and covers the client-originated feedback capability.

**Do**, the root goal is to give feedback on a student's model, decomposed into writing a comment, linking that comment to a specific part of the model, reviewing feedback received, and sharing the result as a file.

**Be**, the capability must not disturb the existing system, must be simple to use, and must follow the conventions already established in the interface. These are carried into section 9 as NF-1 to NF-4.

**Feel**, staff should not find the feature stressful to use. The negative emotional goal is that a student receiving feedback should not feel scolded; this is what the model is guarding

against, and it motivates FB-2 in particular, since feedback anchored to a specific goal reads as directed at the work rather than at the person.

**Who**, reviewer and student. Both are served by the same interface, as noted in section 4.

### 3. Requirement sources

The client's original brief covered the feedback capability only, described in general terms; the team determined what that capability should consist of. The remaining items were proposed by the team and put to the client for a decision.

| Requirement group | Origin | Status |
|---|---|---|
| Feedback on models | Client | Implemented |
| Interface clarity improvements | Team proposal | Adopted (DS-1 in part); approved by the client on 15 Oct |
| Grading | Team proposal | Not required (client, Week 5); kept on its own branch: see 8.1 |
| Feedback status tracking | Team proposal | Not adopted: see 8.2 |
| Guided tour for new users | Team proposal | Adopted; approved by the client on 15 Oct: see 7 |
| Model version history | Team proposal | Adopted; approved by the client on 15 Oct: see 7 |

### 4. Personas and roles

Two roles use the feedback capability: a reviewer marking a submitted model, and a student reading what was said about theirs. Both use the same application; the interface does not authenticate or distinguish them. This follows from the file-based architecture and is a deliberate constraint rather than an omission.

#### Priya Nair: sessional tutor, software engineering course

*"I need to say exactly which goal is unclear: not just 'good model, see notes.'"*

- **Context:** marks a folder of thirty to forty student model files in one sitting, in one browser tab, straight after a submission deadline.
- **Goal:** leave comments tied to specific goals in the diagram, quickly enough to get through the whole class.

- **Frustration:** generic PDF or email comments are disconnected from the diagram, so students cannot tell which box a note refers to.
- **Tech comfort:** confident browser user, not a developer. Will not tolerate a login just to leave a comment.

#### Alex Chen: second-year student

*"I just want to click a comment and see which goal it's talking about."*

- **Context:** builds a motivational model for an assignment, submits the file, later reopens the same file to check for feedback.
- **Goal:** know at a glance whether the model has been reviewed, and see exactly which part of it each comment is about.
- **Frustration:** feedback that is not anchored to the diagram forces guesswork about which goal needs work.
- **Tech comfort:** comfortable in the editor from regular class use; expects it to just open the file, with no separate viewer.

Priya and Alex were written for the feedback capability in Sprint 1. The guided tour and version history are written for the Sprint 2 personas, Mia, a first-time student, and Daniel, a requirements engineer who revises models across workshops. All personas: Personas.

### 5. Client-originated requirements, Feedback

#### FB-1 Entering staff mode

As a reviewer, I want to switch into staff mode from the welcome screen without changing what Create and Open do, so that I can start a review without disrupting the normal workflow.

As a reviewer, I want to confirm I am teaching staff and type my name once per browser tab, so that I am not asked again for every file I open in the same session.

As a reviewer, I want my session to end automatically on refresh or tab close, so that a shared or public computer never stays signed in as me.

Reviewer identity is held in React state only. sessionStorage and localStorage were rejected because they survive a refresh. The confirmation dialog is a workflow gate, not access control: anyone holding the file can open it normally through Open Model, and nothing in the code treats it otherwise, so a real check can replace it without touching the rest.

#### FB-2 Writing and managing feedback

As a reviewer, I want to add a new comment box with one click, so that I can capture feedback as fast as I read the model.

As a reviewer, I want each comment stamped with my name and the date automatically, so that a student always knows who reviewed their work and when.

As a reviewer, I want to edit or delete a comment I have written, so that I can fix or retract feedback before saving.

As a reviewer, I want to be stopped at twenty comments on one model, so that feedback stays readable and every comment keeps a distinct place in the layout.

#### FB-3 Linking feedback to goals

As a reviewer, I want to arm a comment and then click a goal, in the hierarchy or on the rendered diagram, to link them, so that my feedback points at something concrete instead of a vague area of the model.

As a reviewer, I want to click an already-linked goal again to unlink it, so that I can correct a mis-click without deleting and rewriting the whole comment.

As a reviewer, I want to leave linking mode with Esc or the same button, so that I never get stuck in a mode I have forgotten I am in.

Arrows are drawn with a dash pattern distinct from the model's own association edges, and are routed around other shapes so they do not lie across goal labels.

#### FB-4 Reading feedback

As a reviewer or student, I want to select one comment and see only its arrows, so that I can tell exactly which goals it is about when several comments are on screen at once.

As a student, I want to open my model and immediately see whether it has been reviewed, so that I know there is feedback worth reading.

As a student, I want the feedback panel to be read-only when I open it, so that I cannot accidentally edit or delete my tutor's comments.

As a student, I want a comment that references a goal I have since deleted to still display clearly, as (deleted goal), so that I am not confused by a silently broken link.

Comments use the blue already established throughout AMMBER, at the client's request. Selection is what identifies which comment an arrow belongs to, so colour carries no information of its own; see 8.3.

#### FB-5 Saving and persistence

As a reviewer, I want my comments saved inside the same model file the student already uses, so that there is no separate system to manage or lose track of.

As a reviewer, I want a file I have never opened in staff mode to be distinguishable from one I have reviewed and left empty, so that "not yet marked" is never confused with "marked, no notes."

As a user with existing model files, I want those files to keep working, so that adopting the new version costs me nothing.

The save format adds an optional key to the existing .json. Files written before this capability open unchanged; the block is validated and a malformed one is ignored rather than blocking the model from loading. A student re-saving a reviewed model cannot downgrade its reviewed state.

### 6. Team-proposed requirements: adopted

#### DS-1 Distinguishing the primary action buttons

As a user, I want the primary actions to be distinguishable at a glance, so that I do not trigger a destructive action by mistake.

Status: partially adopted. Hover colours were added to Reset, Export and Save and demonstrated to the client, who judged the colour change unnecessary. The buttons were returned to the blue already in use (a95e6f0), while the dropdown hover feedback introduced alongside them was judged useful and kept.

#### DS-2 Indicating the drop target during a drag

As a user rearranging the hierarchy, I want to see where a dragged node will land, so that I do not reparent it accidentally.

Status: adopted following client review.

#### DS-3 Keyboard shortcuts

As a user who models often, I want keyboard shortcuts for common actions, so that I do not have to reach for the mouse.

Ctrl+E then P or S exports directly as PNG or SVG. Shortcuts do not fire while a text field has focus.

Status: adopted; approved by the client on 15 Oct.

#### DS-4 Empty-state guidance

As a new user, I want an empty canvas and an empty hierarchy to tell me what to do next, so that I can tell an empty screen from a broken one.

Status: adopted; approved by the client on 15 Oct.

#### DS-5 Papers as a modal

As a user, I want the Papers page to open over the editor and close back to it, so that I do not lose my place in the model.

Status: adopted; approved by the client on 15 Oct.

#### DS-6 Confirmation after an action

As a user, I want a short notification after save, duplicate and export, so that I know the action worked.

Status: adopted; approved by the client on 15 Oct.

## 7. Team-proposed requirements: adopted in Weeks 7–12

#### VH-1 Model version history

As a user, I want a checkpoint saved automatically whenever I save the model, so that I can go back to an earlier state without having kept copies myself.

As a user, I want to save a named checkpoint deliberately, so that I can mark a point I know I may want to return to.

As a user, I want to see the list of saved versions and restore one, so that I can recover from a change I regret.

As a user, I want to rename or delete a saved version, so that the list stays meaningful as it grows.

Origin: team proposal. Status: adopted; approved by the client on 15 Oct. The persistence concern recorded earlier was resolved by saving checkpoints inside the model's own .json file, so nothing is held in the browser or on a server and NF-1 holds. This is the difference from status tracking (8.2), which needed a store outside the file. Detailed stories and acceptance criteria: US-4 to US-8 on Requirements and Architecture: Guided Tour and Version History.

#### GT-1 Guided tour for first-time users

As a first-time user, I want a walkthrough of the main interface, so that I can start work without external instruction.

- The tour introduces the main editing interface and the principal actions
- The tour can be skipped at any step
- The tour can be restarted deliberately

Status: adopted; approved by the client on 15 Oct. The tour opens on entering the editor, can be turned off with ?tour=off and restarted from the Guide button, so no state is persisted and NF-1 holds. Detailed stories and acceptance criteria: US-1 to US-3 on Requirements and Architecture: Guided Tour and Version History.

## 8. Not adopted and withdrawn

#### 8.1 Grading: withdrawn, not required

As a reviewer, I want to record a mark alongside my comments, so that the student receives the assessment and its justification together.

Origin: team proposal. Grading was written into the feedback capability before the client had asked for it: an overreach the team recognised: and was taken back out (9fbc8ee). Rather than discard the work, the version carrying it was copied to its own branch, so the feedback branch holds feedback alone and grade-system holds feedback with grading.

The design records a total score and its maximum, any number of optional named component scores not required to sum to the total, and an overall comment capped at 500 characters. No marking scheme is assumed, since AMMBER is used across subjects that mark differently.

Status: not required. On reviewing the prototype in Week 5, the client confirmed that grading is not needed; the reasoning is recorded in the client meeting log. The work is kept on the grade-system branch only as a record and is outside the project scope.

#### 8.2 Feedback status tracking: not adopted

As a subject coordinator, I want to see which models have been reviewed, so that I can identify students still waiting.

Origin: proposed by a team member, implemented in full and demonstrated to the client. The implementation divided into two workflows: submission, where a model is sent through a form collecting its name, description and submitter and listed in a new view; and review, where a record is opened back into the editor and, on submission of a grade, updated with the feedback and score. Records were persisted in the browser through IndexedDB.

Status: not adopted. Demonstrated at the Week 6 client meeting. The client indicated it was not a required capability and that they did not want a persistence layer introduced into AMMBER: wherever submitted models are held, the data becomes something that has to be maintained, which is not what the client wants the tool to be. The team abandoned the design rather than carrying it forward; the work remains on its branch as the record of the decision.

Manual verification against five scenarios was carried out before the demonstration. Four passed; the fifth surfaced a defect in which annotations in some modules did not display correctly for a student opening a graded model. The defect was recorded and the work was abandoned before it was repaired.

#### 8.3 Withdrawn after being built

Recorded so they are not re-proposed by accident.

| Capability | Commit | Reason |
| --- | --- | --- |
| Numeric grading inside feedback | 9fbc8ee | Built before the client asked for it; taken out and |

| | | |
|---|---|---|
| | | preserved on the grade-system branch. See 8.1. |
| Annotated export of the model | e751b88 | Feedback is read in the editor rather than exported as a separate artefact. |
| A distinct colour per comment | 4caba85 | Withdrawn at the client's request. Selection already identifies the active comment, and a new palette would have broken the interface conventions the client wanted kept. |
| Real authentication behind staff mode | – | The dialog is a workflow gate, not a security control. A real check can replace it without touching the rest. |

**9. Non-functional requirements**

| ID | Requirement | Source |
|---|---|---|
| NF-1 | No network requests; all state held client-side | Client: existing architecture |
| NF-2 | Existing .json model files remain readable | Client |
| NF-3 | The capability follows the interface conventions already established in AMMBER | Client |
| NF-4 | New capability isolated from existing code where practical, to limit regression risk | Team |
| NF-5 | Malformed persisted data must not prevent a model from loading | Team |

Non-functional requirements specific to the guided tour and version history are on [Requirements and Architecture: Guided Tour and Version History](#).